arXiv:2311.01282v4 [cs.LG] 5 Jan 2024

FLASHDECODING++: FASTER LARGE LANGUAGE MODEL INFERENCE ON GPUS

Ke Hong[†]
Tsinghua University
& Infinigence-AI

Guohao Dai[†✉]
Shanghai Jiao Tong University
& Infinigence-AI

Jiaming Xu[†]
Shanghai Jiao Tong University
& Infinigence-AI

Qiuli Mao
Tsinghua University
& Infinigence-AI

Xiuhong Li
Peking University

Jun Liu
Shanghai Jiao Tong University
& Infinigence-AI

Kangdi Chen
Infinigence-AI

Yuhan Dong
Tsinghua University

Yu Wang[✉]
Tsinghua University

✉daiguohao@sjtu.edu.cn, daiguohao@infini-ai.com, yu-wang@tsinghua.edu.cn

ABSTRACT

As the Large Language Model (LLM) becomes increasingly important in various domains, the performance of LLM inference is crucial to massive LLM applications. However, the following challenges still remain unsolved in accelerating LLM inference: (1) Synchronized partial softmax update. The softmax operation requires a synchronized update operation among each partial softmax result, leading to ∼20% overheads for the attention computation in LLMs. (2) Under-utilized computation of flat GEMM. The shape of matrices performing GEMM in LLM inference is flat, leading to under-utilized computation and >50% performance loss after padding zeros in previous designs (*e.g.,* cuBLAS, CUTLASS, etc.). (3) Performance loss due to static dataflow. Kernel performance in LLM depends on varied input data features, hardware configurations, etc. A single and static dataflow may lead to a 50.25% performance loss for GEMMs of different shapes in LLM inference.

We present *FlashDecoding++*, a fast LLM inference engine supporting mainstream LLMs and hardware back-ends. To tackle the above challenges, *FlashDecoding++* creatively proposes: **(1) Asynchronized softmax with unified max value.** *FlashDecoding++* introduces a unified max value technique for different partial softmax computations to avoid synchronization. Based on this, the fine-grained pipelining is proposed.**(2) Flat GEMM optimization with double buffering.** *FlashDecoding++* points out that flat GEMMs with different shapes face varied bottlenecks. Then, techniques like double buffering are introduced. **(3) Heuristic dataflow with hardware resource adaptation.** *FlashDecoding++* heuristically optimizes dataflow using different hardware resource (*e.g.,* Tensor Core or CUDA core) considering input dynamics.Due to the versatility of optimizations in *FlashDecoding++*, *FlashDecoding++* can achieve up to **4.86**× and **3.93**× speedup on both NVIDIA and AMD GPUs compared to Hugging Face implementations. *FlashDecoding++* also achieves an average speedup of **1.37**× compared to state-of-the-art LLM inference engines on mainstream LLMs.

[†]These authors contributed equally to this work.

[‡]Prof. Guohao Dai is the Chief Scientist at Infinigence-AI, Ke Hong, Jiaming Xu, Qiuli Mao, and Jun Liu are interns at Infinigence-AI.

[✉]Prof. Guohao Dai and Prof. Yu Wang are the corresponding authors of this paper.

1 Introduction

As the Large Language Model (LLM) achieved unprecedented success in various domains [2, 3, 4, 5], the LLM inference workload is skyrocketing. For example, OpenAI reports that GPT-4 inference with 8K context length costs $0.03 per 1K input tokens and $0.06 per 1K output tokens [6]. Currently, OpenAI has 180.5 million users and receives over 10 million queries per day [7]. Consequently, the cost to operate OpenAI's model like ChatGPT is approximately $7 million per day for the necessary computing hardware [8]. Thus, optimizations on LLM inference performance will have a huge impact considering massive LLM inference scenarios. Many recent works have proposed techniques to accelerate LLM inference tasks, including DeepSpeed [9], FlexGen [10], vLLM [11], OpenPPL [12], FlashDecoding [13], TensorRT-LLM [14], and etc [15, 16, 17, 12].

The LLM inference task generates tokens (*e.g.,* words) from the input sequence autoregressively, and can be organized into two typical phases: the *prefill* phase and the *decode* phase. The *prefill* phase generates the first token by processing the input prompt, and previous research (*e.g.,* FlashAttention [18, 19]) optimizes latency for this phase. The *decode* phase generates the following tokens sequentially, and many works [9, 10, 11, 15, 13, 14, 20] focus on improving the throughput of generating tokens (*i.e.,* reducing latency of each token). The *prefill* phase dominates total time for scenarios of long-sequence input or generating short outputs [21, 22], while the *decode* phase constitutes a significant portion of the time when processing long output sequences [23].

Figure 2 shows the main dataflow of the LLM inference with one transformer layer for both the *prefill* phase and the *decode* phase. A transformer layer can be divided into linear GEMM (General Matrix Multiplication) operations (*e.g., K, Q, V, O* weight projection and the feedforward) and the attention/softmax computation. For the attention computation, a softmax operation is adopted for a row in the attention matrix. To improve the parallelism, previous designs [18, 13] divide the attention matrices into smaller tiles and rows are also split to compute partial softmax results. A synchronized softmax operation is adopted to update previous partial softmax results when a new partial softmax result is calculated. Such a **synchronized partial softmax update** accounts for 18.8% for the attention computation of Llama2-7B inference according to our profiling on NVIDIA Tesla A100 GPU with 1024 input length, resulting in the first challenge for accelerating LLM inference. Secondly, **the computation resources is under-utilized for the flat GEMM operation** during the *decode* phase. Because the *decode* phase sequentially generates tokens, the linear GEMM operation tends to be flat-shape (even turning into the GEMV (General Matrix-Vector Multiplication) operation when the batch size is 1). For the small batch size (*e.g.,* 8), previous designs [24, 25] pad the matrix with zeros to perform GEMMs of larger sizes (*e.g.,* 64), leading to over 50% computation under-utilization. Thirdly, **the performance of LLM inference suffers from the static dataflow** considering input dynamics and hardware configuration. For example, the small batch size makes the *decode* phase of LLM inference memory-bounded and the large batch size makes it compute-bounded. A single and static dataflow may lead to 50.25% performance loss for GEMMs of different shapes in LLM inference.

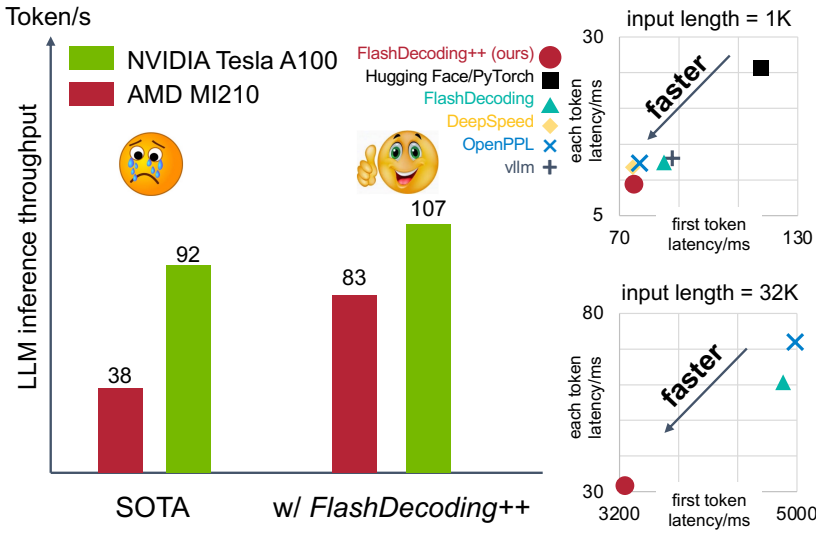

Figure 1: Overview of comparison between *FlashDecoding++* and state-of-the-art designs. The results in the figure are reported with Llama2-7B model [1]. The left is with batch size=1 and input length=1K, and TensorRT-LLM and Hugging Face are the SOTA baseline for NVIDIA/AMD according to our experimental results. The right shows the comprehensive comparison of both first token latency and each token latency.

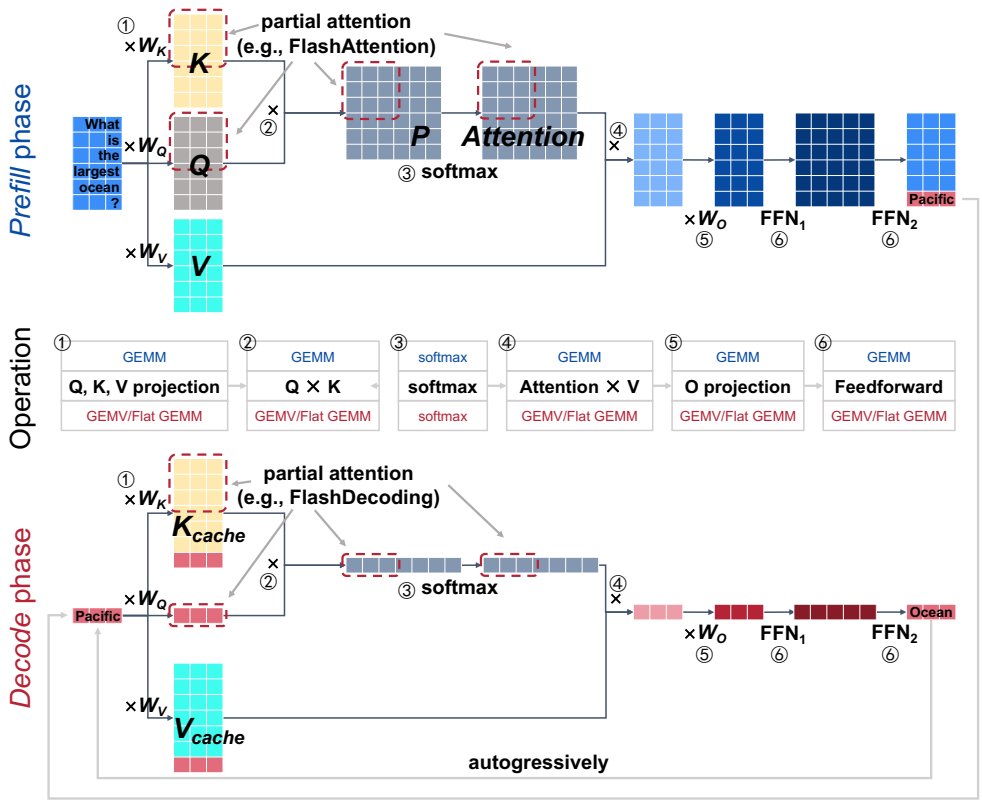

Figure 2: Overview of Large Language Model inference dataflow. We show the dataflow comparison between the *prefill* phase and the *decode* phase. The *prefill* phase mainly involves the GEMM operation, while the *decode* phase mainly involves the GEMV/Flat GEMM operation.

To tackle these challenges and enable a faster Large Language Model (LLM) inference, we present *FlashDecoding++* in this paper. *FlashDecoding++* creatively proposes the following contributions:

- **Asynchronized softmax with unified max value.** *FlashDecoding++* leverages a unified max value for different partial softmax computations. Each partial softmax result can be processed individually without synchronized update.
- **Flat GEMM optimization with double buffering.** *FlashDecoding++* only pads the matrix size to 8 rather than 64 in previous designs for flat-shaped GEMM to improve computation utilization. We point out that flat GEMMs with different shapes face varied bottlenecks, and further improve the kernel performance with techniques like double buffering.
- **Heuristic dataflow with hardware resource adaption.** *FlashDecoding++* takes both input dynamics and hardware configurations into consideration and dynamically applies kernel optimization for the LLM inference dataflow.

Because of the versatility of optimizations, the effectiveness of *FlashDecoding++* can be proved on both NVIDIA and AMD GPUs. *FlashDecoding++* achieves up to **4.86**× and **3.93**× speedup on both NVIDIA and AMD GPUs compared with Hugging Face implementations, respectively. Our extensive results show that *FlashDecoding++* achieves an average of **1.37**× speedup compared with FlashDecoding [13], a state-of-the-art LLM inference engine on various LLMs (*e.g.,* Llama2, ChatGLM2, etc.).

The rest of this paper is organized as follows. Section 2 introduces preliminaries of LLMs and related works on LLM inference acceleration. Our three techniques, the asynchronized softmax with unified max value, the flat GEMM optimization with double buffering, and the heuristic dataflow with hardware resource adaption are detailed in Section 3, 4, and 5, respectively. Section 6 presents the evaluation results. Related works on LLM inference are introduced in Section 7, and Section 8 concludes the paper.

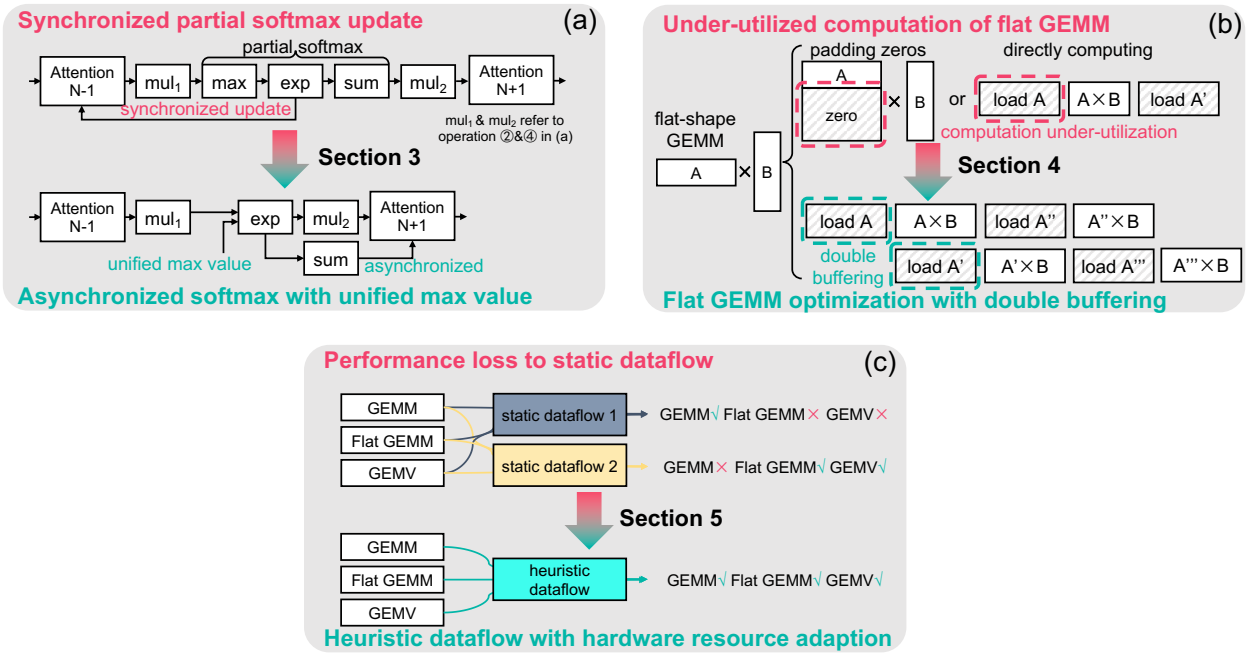

Figure 3: *FlashDecoding++* proposes three solutions for corresponding challenges in Large Language Model inference. (a) *FlashDecoding++* proposes the asynchronized softmax with unified max value technique, avoiding synchronized update to previous partial attention results. (b) *FlashDecoding++* optimizes flat GEMM by improving computation utilization. (c) *FlashDecoding++* heuristically optimizes dataflow.

2 Background

2.1 LLM Inference Dataflow Overview

The task of LLM inference is to generate tokens from the input sequence, which can be used to complete a sentence or answer a question. An overview of the LLM inference dataflow is shown in Figure 2. As we can see, the LLM inference dataflow can be organized into two typical phases with similar operations: one *prefill* phase and several *decode* phases. The *prefill* phase "understands" the input sequence (*i.e.,* "What is the largest ocean?"). Each token (we set one word as a token in Figure 2 is encoded as an embedding vector, and the input sequence is organized into a matrix. The main output of the *prefill* phase is a new token, which is predicted to be the next token after the input sequence (*i.e.,* "Pacific" in this figure). The *decode* phase "generates" the output sequence (*i.e.,* "Pacific", "Ocean", etc.) The output token of the *prefill* phase is taken as the input of the *decode* phase. The *decode* phase is executed autogressively, and each output token is used as the input token for the next The *decode* (*e.g.,* "Ocean" is further used as the input).

2.2 Operations in LLM Inference

The main operations in LLM inference are depicted as operation ① to ⑥ in Figure 2, including the linear projection (① and ⑤), the attention (②, ③, and ④), and the feedforward network (⑥). For simplicity, operations like position embedding [26], non-linear activation [27, 28, 29], mask [26], and others are not shown in the figure. Operations in the *prefill* phase and the *decode* phase are different in the shape of data. Because only one token (batch size=1) or few tokens (batch size>1) are processed at one time, **input matrices in the *decode* phase are flat-shape matrices or even vectors.**

Linear Projection. The linear projection performs as the fully connected layer, multiplying the input with weight matrices (*i.e.,* W_K, W_Q, W_V, W_O, called K, Q, V projection and O projection). For the *prefill* phase, the K, Q, V projection generates matrices K, Q, V. For the *decode* phase, the K, Q, V projection generates three corresponding vectors and concatenated with K and V (*i.e.,* KVcache, yellow and light blue in Figure 2 in the *prefill* phase.

$$softmax(Q \times K^T) \times V \tag{1}$$

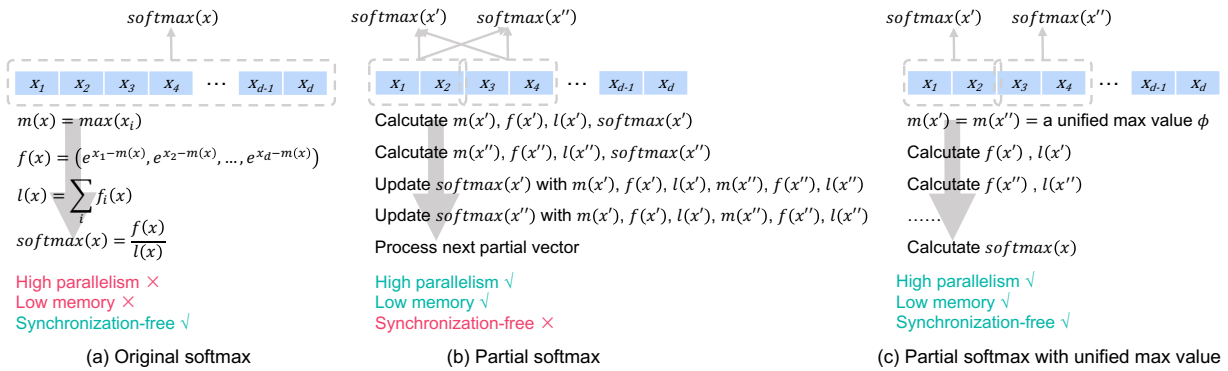

Figure 4: Comparison of different softmax computation schemes. (a) Softmax computation for the whole vector. (b) Computing partial softmax for each partial vector, and a synchronized update operation is required for all partial softmax results. (c) Computing partial softmax using a unified max value, and each partial vector is processed individually without synchronized update.

Attention. The attention operation is mainly divided into three operations (② to ④ $Q \times K$, $softmax$, $Attention \times V$), as shown in Eq. (1). For $P = Q \times K^T$, the softmax operation is performed for each row of the result matrix of P. The detailed softmax computation is shown in Figure 4(a). The maximum value $m(x)$ is first calculated. The exponent of each element divided by $e^{m(x)}$, $f(x)$, is then processed. These exponents are normalized to the summation of all exponents (*i.e.,* $l(x)$) to get the softmax result.

Feedforward Network. The feedforward network primarily comprises two fully connected layers. The first one (⑥ FFN_1) expands the feature dimensions to enhance the representational capacity. The second one (⑥ FFN_2) restores the feature dimensions and serves as the output layer.

2.3 Attention Optimization

The softmax operation shown in Figure 4(a) requires all global data to be calculated and stored before it can proceed. This results in high memory consumption and low parallelism. Latter works propose the partial softmax technique to reduce memory consumption [18, 19] or improve parallelism [13]. Figure 4(b) shows the diagram of the partial softmax operation. The main idea is to divide the vector x into partial vectors (*i.e,* x' and x''). The partial softmax results of x' and x'' are calculated separately according to Figure 4(a), and then synchronously updated by each other. The detailed computation of this synchronized update is shown in Equation (2). With the implementation of partial softmax, we can achieve efficient parallelism of computation while reducing memory cost for attention computation.

$$
\begin{aligned}
m(x) &= max(m(x'), m(x'')) \\
f(x') &= e^{m(x')-m(x)} f(x') \\
f(x'') &= e^{m(x'')-m(x)} f(x'') \\
l(x) &= f(x') + f(x'') \\
softmax([x', x'']) &= [f(x'), f(x'')] \div l(x)
\end{aligned} \tag{2}
$$

However, since the partial softmax needs to be updated according to other partial softmax results, it unavoidably introduces data synchronization operations. According to our profiling result, such a synchronized update operation leads to 18.8% overheads in the attention computation for Llama2-7B inference on NVIDIA Tesla A100 GPU with 1024 input length.

3 Asynchronized Softmax with Unified Maximum Value

Motivation. The partial softmax operation requires synchronization among different partial vectors, leading to ∼20% overheads of the attention operation. As is shown in Figure 3(a), the synchronization is required after the maximum value of the partial vector is calculated. The maximum value is used to update previous partial softmax (*i.e.,* recompute previous attention) results. Thus, to reduce synchronization overheads, **the key problem to be solved is how to compute each partial softmax result without requiring results from other partial softmax computation.**

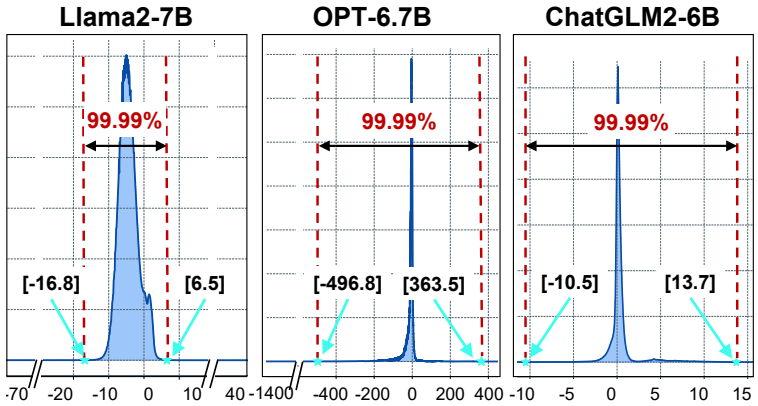

Figure 5: The statistical distribution of x_i (elements in the input vectors of softmax) in typical LLMs with different inputs.

Challenge. The reason that synchronization is required lies in that the maximum value of each partial vector is different. The maximum value is used to avoid overflow of the exponent operation ($f(x)$ in Figure 4(a)), and exponents are summed ($l(x)$ in Figure 4(a)) as the denominator of the softmax operation. Such a non-linear operation on each partial maximum value makes the synchronization among each partial softmax computation unavoidable.

Analysis and Insights. According to the formula of softmax computation, the maximum value is used as the scaling factor for both the numerator and the denominator (*i.e.,* $f(x)$ and $l(x)$ in Figure 4(a)). Our key insight is, **the scaling factor can be an arbitrary number** rather than using the maximum value mathematically, shown in Equation (3). When we set $\phi = 0$, it becomes the original softmax computation [30].

$$\begin{aligned} softmax(x) &= \frac{[e^{x_1-m(x)}, ..., e^{x_d-m(x)}]}{\sum_i e^{x_i-m(x)}} \\ &= \frac{[e^{x_1-\phi}, ..., e^{x_d-\phi}]}{\sum_i e^{x_i-\phi}}, \forall \phi \in \mathbb{R} \end{aligned} \tag{3}$$

However, the scaling factor cannot be an arbitrary number considering the overflowing of the exponent computation. For the case where $x_i \gg \phi$, $e^{x_i-\phi}$ overflows and cannot be represented using a fix-width floating point number (*e.g.,* `float32` for exponent results in current LLM engines). For another case where $x_i \ll \phi$, $e^{x_i-\phi} \rightarrow 0$, leading to precision loss. Thus, a proper scaling factor ϕ should be carefully selected to avoid the two cases above. Figure 5 shows the statistical distribution of x_i (elements in the input vectors of softmax) in typical LLMs with different inputs [31]. Our key insight is, $> 99.99\%$ x_i **are within a certain range**. Specifically, for Llama2-7B, we have $-16.8 < x_i < 6.5$ for $> 99.99\%$ x_i. Because e^{b-a} and e^{a-b} can be represented by a `float32` format, we can set $\phi = a$ in Equation (3). For OPT-6.7B, we do not apply the technique in this section because of the large range in Figure 5.

Approach: Asynchronization. Based on the insights above, each partial softmax computation shares a unified maximum value, ϕ. After the softmax operation, an inner product operation is executed between the softmax result and a column of V (*i.e.,* v). Assume that the input vector x can be divided into p partial vectors, $x = [x^{(1)}, ..., x^{(p)}]$ ($v = [v^{(1)}, ..., v^{(p)}]$ correspondingly), we have:

$$\begin{aligned} \langle softmax(x), v \rangle &= \frac{\sum_i e^{x_i-\phi} \cdot v_i}{\sum_i e^{x_i-\phi}} \\ &= \frac{\sum_{j=1}^{p} \sum_{i=1}^{d/p} e^{x_i^{(j)}-\phi} \cdot v_i^{(j)}}{\sum_{j=1}^{p} \sum_{i=1}^{d/p} e^{x_i^{(j)}-\phi}} \end{aligned} \tag{4}$$

The inner accumulation in both the numerator and the denominator only take the partial vectors $x^{(j)}$ and $v^{(j)}$ as input, thus they can be processed asynchronously and individually. The outer accumulation is only processed after all partial vectors are processed. As we can see in Figure 4(c), each $f(x^{(j)})$ is calculated individually, and $softmax(x)$ is calculated after all $x^{(j)}$ is calculated.

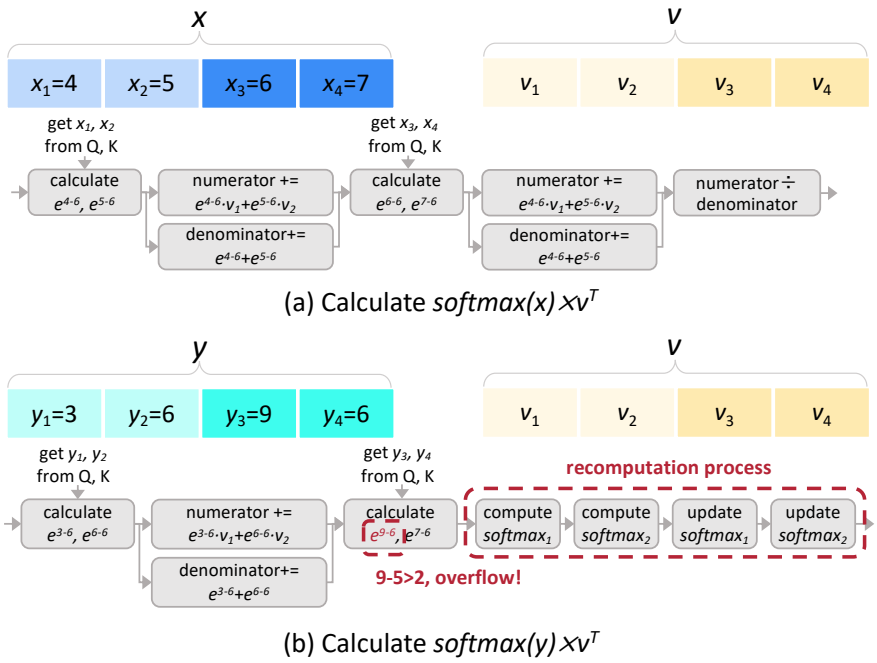

Figure 6: Example of asynchronized partial softmax computation. (a) Each partial softmax result is process individually without the synchronized update. (b) The recomputation process for all parital softmax computation is required when overflow happens.

Approach: Recomputation. Without loss of generality, we assume $a < x_i - \phi < b$ for each x_i to ensure precision and avoid overflow. Then, the partial softmax operation is processed individually. However, when $x_i - \phi \leq a$ or $x_i - \phi \geq b$, the asynchronized partial softmax computation is terminated for the vector x where x_i belongs to. The softmax is then recomputed using the synchronized partial softmax scheme (used in FlashAttention [18, 19] and FlashDecoding [13]) shown in Figure 4(b). Such a recomputation scheme avoids overflow while introducing negligible overheads based on the statistical data shown in Figure 5.

Example. Figure 6 shows an example of the asynchronized softmax scheme. We set $a = -3, b = 3, \phi = 6$. Two vectors x and y are calculated from $Q \times K^T$ in Equation (1), and are divided into 2 partial vectors. We omit the process from $Q \times K^T$ to these partial vectors. For each x_i, we have $a < x_i - \phi < b$, we process $e^{x_1-\phi} \cdot v_1 + e^{x_2-\phi} \cdot v_2$ and $e^{x_1-\phi} + e^{x_2-\phi}$ for the first partial vector of x using two asynchronized threads. Then, each thread moves to the next partial vector for the corresponding computation (*i.e.,* $e^{x_3-\phi} \cdot v_3 + e^{x_4-\phi} \cdot v_4$ and $e^{x_3-\phi} + e^{x_4-\phi}$). Two threads are synchronized when all partial vectors are processed, and perform the division operation in Equation (4). For y, the first partial vector is processed similarly. However, we find that $y_3 - \phi > b$, then two threads are terminated and the first thread recomputes all partial vectors according to the synchronized partial softmax scheme in Figure 4(b).

4 Flat GEMM Optimization with Double Buffering

Motivation. The process of the *decode* phase is mainly composed of GEMV (batch size=1) or flat GEMM (batch size>1) operation. Without loss of generality, GEMV/GEMM operations can be represented using M, N, K, where the sizes of two multiplied matrices are $M \times K$ and $K \times N$. Previous LLM inference engines utilize Tensor Core to accelerate these operations using libraries like cuBLAS [24] and CUTLASS [25]. Although modern Tensor Core architectures [32] process GEMM with $M = 8$, these libraries usually tile the $M-$dimension to 64 to hide memory latency. However, for GEMV or flat GEMM operations in the *decode* phase, we usually have $M \ll 64$ and the $M-$dimension is padded to 64 with zeros. The padding leads to under-utilized computation, and **the key problem is to process GEMV or flat GEMM operations with smaller tiles (*i.e.,* padding to 8 corresponding to modern Tensor Core architectures) in the $M-$dimension**.

Challenge. Processing GEMV or flat GEMM operations is non-trivial when the $M-$dimension is padded to 8. The tiling technique in modern libraries like cuBLAS [24] and CUTLASS [25] can only be applied to the $N-$dimension and

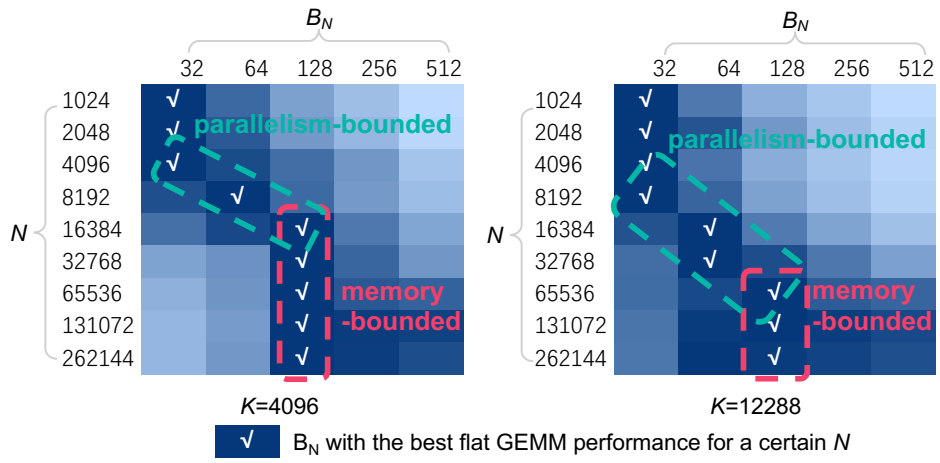

Figure 7: Normalized flat GEMM performance under different $N-$dimension sizes and $N-$dimension tiling sizes. We set $M = 8$ and execute GEMM on the NVIDIA Tesla A100 GPU.

the $K-$dimension. Tiles on the $K-$dimension are processed sequentially in a GPU block to avoid atomic operations during reduction. Tiling on the $N-$dimension affects both parallelism and computation/memory ratio, which are both important for GEMV and flat GEMM acceleration.

Analysis and Insights. Assume that tiling sizes of the $N-$dimension and the $K-$dimension are B_N and B_K, respectively. The computation of each GEMM tile is $2 \times M \times B_N \times B_K$ with total $B = \frac{N \times K}{B_N \times B_K}$ GEMM tiles. The total memory access is $(M \times B_K + B_N \times B_K) \times B + M \times N$. Thus, the computation/memory ratio is:

$$\begin{aligned} &\frac{2 \times M \times B_N \times B_K \times B}{(M \times B_K + B_N \times B_K) \times B + M \times N} \\ =&\frac{2 \times M \times K}{K + \frac{M \times K}{B_N} + M} \end{aligned} \tag{5}$$

On the other hand, the parallelism is $\frac{N}{B_N}$. Thus, the computation/memory ratio shows a positive correlation with B_N while the parallelism shows a negative correlation with B_N, exposing a contradiction on improving the performance of GEMV or flat GEMM. We depict the normalized performance of the flat GEMM in Figure 7 with different N and B_N. Our key insight is, **for the smaller N, the flat GEMM is parallelism-bounded**. There are 108 Streaming Multiprocessors (SMs) in the NVIDIA Tesla A100. $\frac{N}{B_N}$ tends to be a constant (*e.g.,* 128 or 256), which is related to the hardware parallelism (number of SMs). Another key insight is, **for the larger N, the flat GEMM becomes memory-bounded**. The performance of these cases can be improved by hiding memory access latency.

Approach: Double Buffering. In order to hide memory access latency, we introduce the double buffering technique. for the flat GEMM operation. We allocate two separate buffers in the shared memory. The tile in one buffer performs the GEMM operation, while another buffer loads a new tile for the next GEMM operation. Thus, the computation and the memory access are overlapped. We apply such a technique when N is large in our practice.

Example. Figure 8 shows the example of our flat GEMM optimization with double buffering. For $M < 8$, the $M-$dimension is first padded to 8 considering modern Tensor Core architectures. Workloads in the $K-$dimension are processed within one GPU block (*e.g.,* $A_1, A_2, A_3, ...$), while workloads in the $N-$dimension are processed in parallel using different GPU blocks (*e.g.,* $C_1, C_2, ...$). We take GPU Block$_1$ as an example, the first tile for each matrix in the $K-$dimension (*i.e.,* A_1 and B_1) is loaded to the left buffer in the shared memory. Then, the GEMM operation is performed between A_1 and B_1. Consequently, A_2 and B_2 are loaded to the right buffer in the shared memory. The following tiles are processed similarly according to the double buffering scheme.

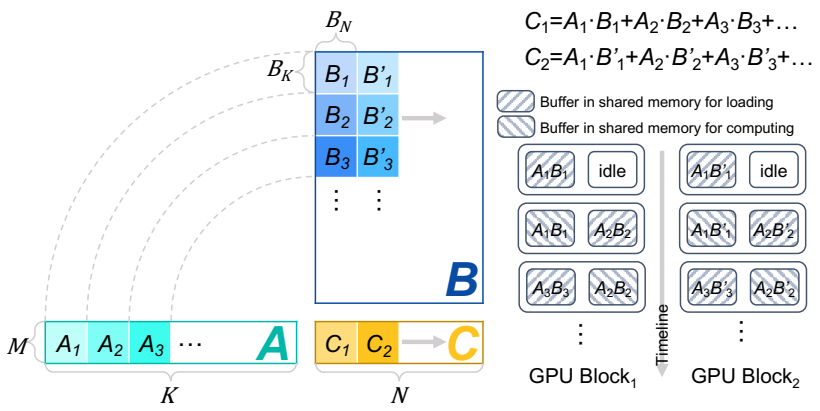

Figure 8: Double buffering for flat GEMM when $N-$dimension is large. The $M-$ dimension is padded to 8 and not tiled.

5 Heuristic Dataflow with Hardware Resource Adaption

Motivation. Although *FlashDecoding++* optimizes the flat GEMM operation in Section 4, it does not cover all operations (even only for GEMMs) in the LLM inference. As mentioned in Figure 2, the shapes of GEMMs in different operations and two phases vary. Thus, the GEMM workload in the LLM inference can be GEMV (batch size=1 for the *decode* phase), flat GEMM (small batch size for the *decode* phase and short sequence length for the *prefill* phase) and conventional GEMM (large batch size or long sequence length for the *prefill* phase). In order to leverage the powerful computational ability of Tensor Core, current frameworks like FasterTransformer [33] and DeepSpeed [9] tend to utilize the highly optimized GEMM implementation from cuBLAS [24] to deal with different workloads. However, the Tensor Core implementation fails with the GEMV workload. The GEMV workload can be optimized by utilizing CUDA Core in previous designs like FastGEMV [34]. For a Llama2-7B linear layer in the *decode* phase, the Tensor Core implementation from cuBLAS only achieves 82.15% of the performance of CUDA Core implementation using FastGEMV on an NVIDIA A100 GPU. On the other hand, using CUDA Core to do the projection on a batchsize=4 decoding input only achieves 49.75% performance compared with the Tensor Core implementation. Thus, in order to approach the optimal computation performance, **a heuristic dataflow is supposed to be exploited in for different workloads.**

Challenge. Although a heuristic dataflow potentially exists in the implementation of different linear workloads, it is challenging to build the mapping from a certain workload to an optimal implementation. In the scenario of LLM inference, there are various factors that influence the implementation performance of linear workloads: (a) Input dynamics. The variety of the batch size and the input sequence length brings dynamic workloads. (b) Model diversity. The linear workload varies with different model structures and sizes. (c) GPU capacities. The relative performance between implementations changes with GPU characteristics, such as memory bandwidth, cache size, and computational ability. (d) Engineering effects. The engineering effort also highly impacts the kernel performance. All these influential factors build a large search space, making it non-trivial to generate an effective mapping between the linear workload and the corresponding optimal implementation.

Analysis and Insights. Although all influential factors form a large search space, the homogeneity of different layers in LLM significantly reduces the search space for operator optimization. Figure 2 shows four linear GEMV/GEMM operations in the *prefill* phase and the *decode* phase, *i.e.,* K, Q, V projection, O projection, and two feedforward operations. Each GEMV/GEMM operation can be can be abstracted as a multiplication between an $(M \times K)$-shaped matrix and a $(K \times N)$-shaped matrix. Our key insight is, **there are only four $[K, N]$ shapes for a certain LLM.** Moreover, M is only related to the input sequence length and the batch size for the *prefill* phase, and the batch size for the *decode* phase. Figure 9(a) shows limited shapes of GEMV/GEMM operations in the LLM inference.

Approach: Decision flow for inflection points. Because only four $[K, N]$ shapes exist for a certain LLM, we use three types of implementations for GEMV/GEMM operations when M varies: FastGEMV for the GEMV and flat GEMM operations (ImplA), our flat GEMM optimization in Section 4 (ImplB), and the CUTLASS [25] libraries optimized for the conventional GEMM (ImplC). Thus, it is important to decide whether applying ImplA or ImplB for a small M, and ImplB or ImplC for a large M. Figure 9(b) shows the decision flow. *FlashDecoding++* profiles the performance of ImplA and ImplB for a certain M, and increases M to find an inflection point M_1 where the performance of ImplB is

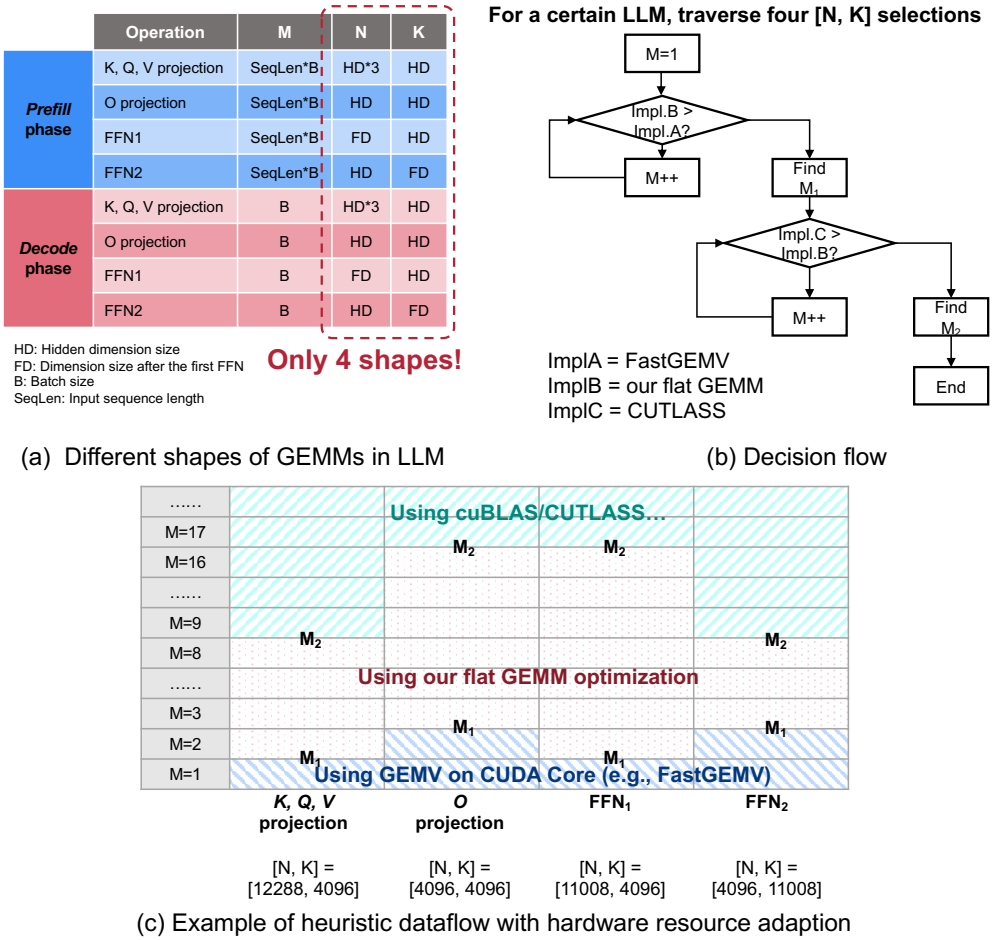

Figure 9: Heuristic dataflow with hardware resource adaption in *FlashDecoding++*. (a) Only four $[N, K]$ shapes exist for a certain LLM. (b) The decision flow. We traverse all $[N, K]$ selections and profile the performance of three representative implementations. M is increased to find two inflection points for runtime heuristic dataflow. (c) *FlashDecoding++* heuristically utilizes Tensor Core/CUDA Core with the corresponding GEMV/GEMM implementation by referring to a lookup table.

better than ImplA. Another inflection point M_2 is found similarly where the performance of ImplC is better than ImplB. Note that each $[N, K]$ gets its individual M_1 and M_2.

Approach: Heuristic dataflow. For the runtime LLM inference, *FlashDecoding++* adopts ImplA using CUDA Core when $M < M_1$, and ImplB/ImplC using Tensor Core when $M_1 \leq M < M_2$/$M_2 \leq M$. Note that the decision flow are executed offline, it does not affect the performance of runtime LLM inference.

Example. Figure 9(c) shows an example of applying the heuristic dataflow for the Llama2-7B model. Four $[N, K]$ shapes are [12288, 4096] for K, Q, V projection, [4096, 4096] for O projection, [11008, 4096] and [4096, 11008] for FFN. For each $[N, K]$, the inflection points are found based on the decision flow in Figure 9(c). Then, a lookup table is formed, and each GEMV/GEMM operation is executed according to corresponding implementations during runtime. In this example, FastGEMV is adopted for the K, Q, V projection when batch size=1 ($M = 1$) for the *decode* phase, and our flat GEMM optimization is applied when batch size=1/input sequence length=8 for FFN_1 ($M = 8$).

6 Evaluation

6.1 Experiments Setup

We evaluate the performance of *FlashDecoding++* on different GPUs with various Large Language Models. We compare the performance with several state-of-the-art LLM inference engines.

Table 1: Hardware Platforms

	NVIDIA		AMD	
GPU	Tesla A100 80 GB CUDA 12.2	RTX3090 24 GB CUDA 11.6	MI210 64GB ROCm 5.7	RX7900XTX 24GB ROCm 5.6
CPU	Intel Xeon Silver 8358P 2.60 GHz	Intel Xeon Gold 6226R 2.90GHz	AMD EPYC 7K62 2.60GHz	Intel Core i9-10940X 3.30GHz

Table 2: Model Configuration

Model	Dimension	Heads	Layers	Context Length
Llama2-7B	4096	32	32	4k
Llama2-13B	5120	40	40	4k
OPT-6.7B	4096	32	32	2k
ChatGLM2-6B	4096	32	32	32k

6.1.1 Hardware Platforms

We evaluate the performance of *FlashDecoding++* and other LLM engines on both NVIDIA and AMD platforms to make a comprehensive comparison. We choose two different GPUs for each platform: Tesla A100 and RTX3090 for NVIDIA, MI210 and RX7900XTX for AMD. We show the detailed configuration in Table 6.1.1.

6.1.2 LLM Engine Baselines

We implement our *FlashDecoding++* using the Pytorch-based front-end with the C++ and CUDA backend for NVIDIA GPUs while ROCm for AMD GPUs. We compare the inference performance in both *prefill* phase and *decode* phase with the following LLM engine baselines: Hugging Face (HF) [35], vLLM [11], DeepSpeed [9], TensorRT-LLM [14], OpenPPL [12], and FlashAttention2/FlashDecoding [19, 13]. These baselines are introduced in Section 7.

6.1.3 Models

We evaluate the performance of *FlashDecoding++* with other LLM inference engines on three typical Large Language Models: Llama2, OPT, and ChatGLM2. Table 6.1.2 shows the detailed configuration of these models. Note that there may be several models in one LLM (*e.g.,* Llama2-7B, Llama2-13B) with different configurations (*e.g.,* number of heads and layers).

- **Llama2** [1] is a mainstream open-source LLM set released by Meta in 2023. It is a collection of pretrained and fine-tuned generative text models ranging in scale from 7B to 70B parameters.
- **OPT** [36], is a suite of decoder-only pre-trained transformers ranging from 125M to 175B parameters released by Meta AI.
- **ChatGLM2** [37] is an open-source LLM supporting bilingual (Chinese-English) chat.

6.2 Comparison with State-of-the-art

We compare *FlashDecoding++* with state-of-the-art LLM inference engines in Figure 10 and Figure 11 on NVIDIA GPUs, Figure 12 and Figure 13 for AMD GPUs. For the *decode* phase, *FlashDecoding++* achieves up to **4.86**× speedup compared with Hugging Face implementations on three LLMs and two GPUs. The average speedup over vLLM, DeepSpeed, TensorRT-LLM, OpenPPL, and FlashDecoding is 1.24×, 1.44×, 1.13×, 1.24×, and 1.21× (**1.37**× on Tesla A100 compared with FlashDecoding), respectively. For the *prefill* phase, *FlashDecoding++* achieves up to 1.40× speedup compared with Hugging Face implementations. The average speedup over DeepSpeed, TensorRT-LLM, OpenPPL, FlashAttention2 and FlashDecoding is 1.05×, 1.06×, 1.08×, 1.09×, and 1.08×, respectively. We also show the *decode* results on two AMD GPUs. Currently, only the original Hugging Face implementation can be executed on AMD GPUs as the baseline. *FlashDecoding++* achieves up to 2.27× and **3.93**× compared with the baseline on RX7900XTX and MI210, respectively.

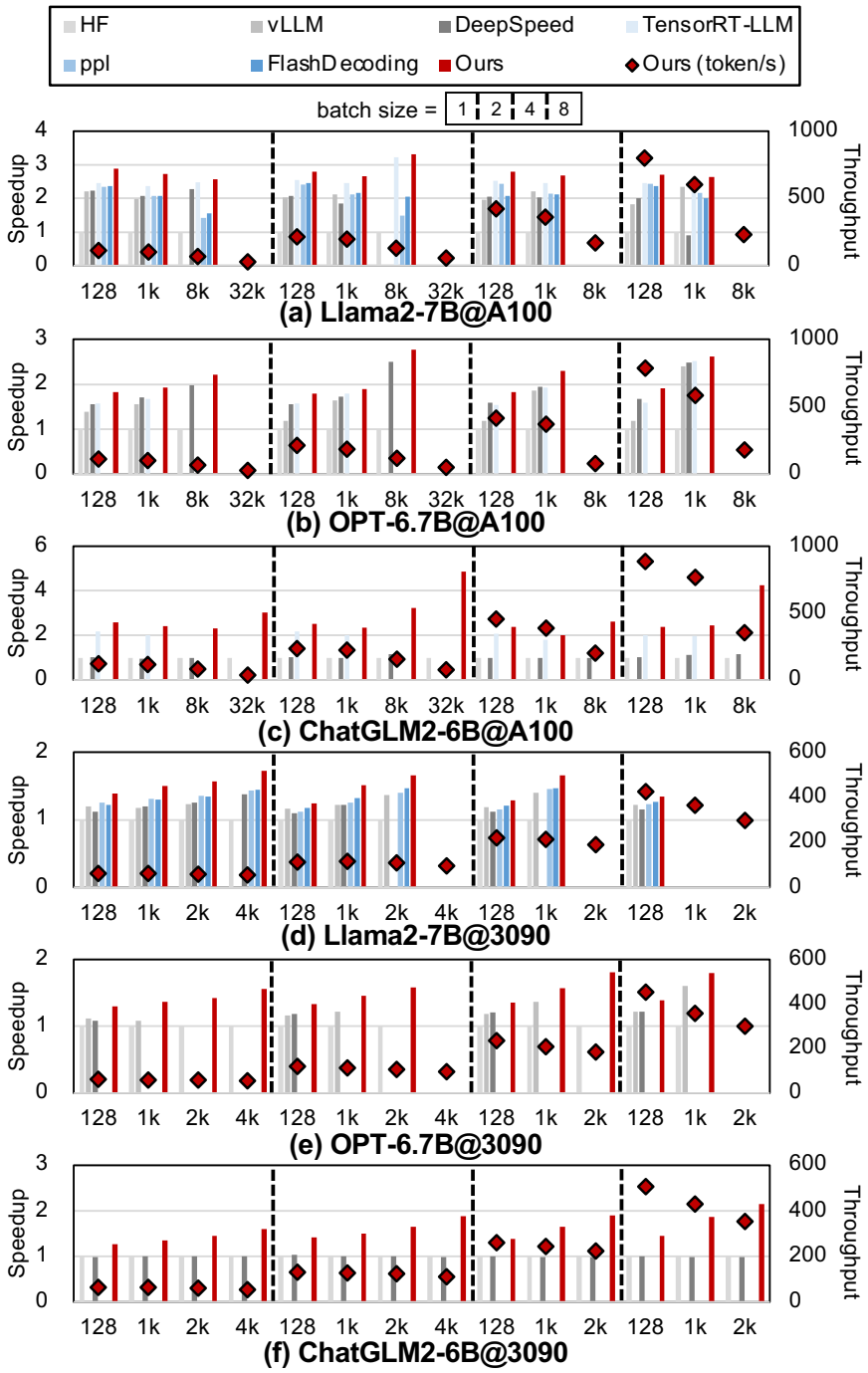

Figure 10: Speedup of the *decode* phase on NVIDIA GPUs. Blank bars represent the model cannot be executed (*e.g.*, OpenPPL does not support OPT-6.7B/ChatGLM2-6B, TensorRT-LLM fails to compile the model with $>$ 8K input length, and etc.)

7 Related Works

Large language model inference acceleration has gained significant attention in recent research, with several notable approaches and techniques emerging in the field. **DeepSpeed** [9] is a comprehensive engine that optimizes both the training and inference phases for LLMs. It achieves robust inference performance through kernel fusion and efficient GPU memory management, with a particular focus on optimizing memory usage for KVcache. **vLLM** [11] improves

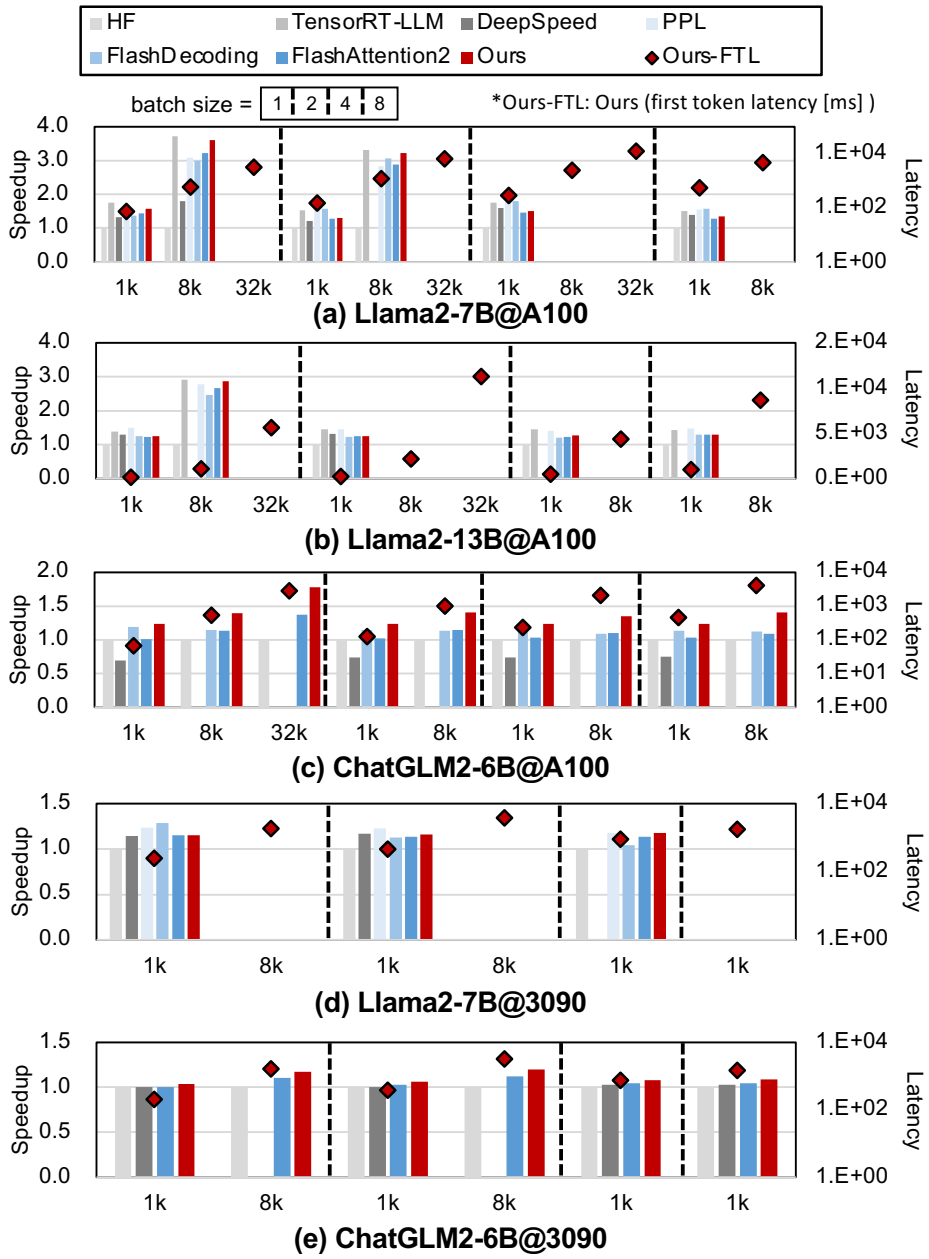

Figure 11: Speedup of the *prefill* phase on NVIDIA GPUs.

GPU memory utilization by efficient memory management techniques and the PageAttention method, leading to increased maximum batch sizes and elevating the upper limit of inference performance. **FlashAttention** [18, 19] optimizes the self-attention computation process during the prefill phase through improved parallelism and workload distribution. **FlashDecoding** [13] is an extension of FlashAttention and enhances the parallelism through spliting K and V, supporting efficient self-attention computation for long sequence during the decode phase. **FasterTransformer** [33] and **OpenPPL** [12] implement large model inference engines using C++ to reduce overhead resulting from kernels scheduling, compared to *Python* implementations. They also employ memory management techniques and kernel fusion to achieve efficient LLM inference. **TensorRT-LLM** [14] is built upon the *TensorRT* [38] and the *FasterTransformer* [33] engine (C++) and incorporates cutting-edge open-source technologies such as *FlashAttention* [18, 19]. Additionally, it enhances its ease of use by providing the *Python API*.

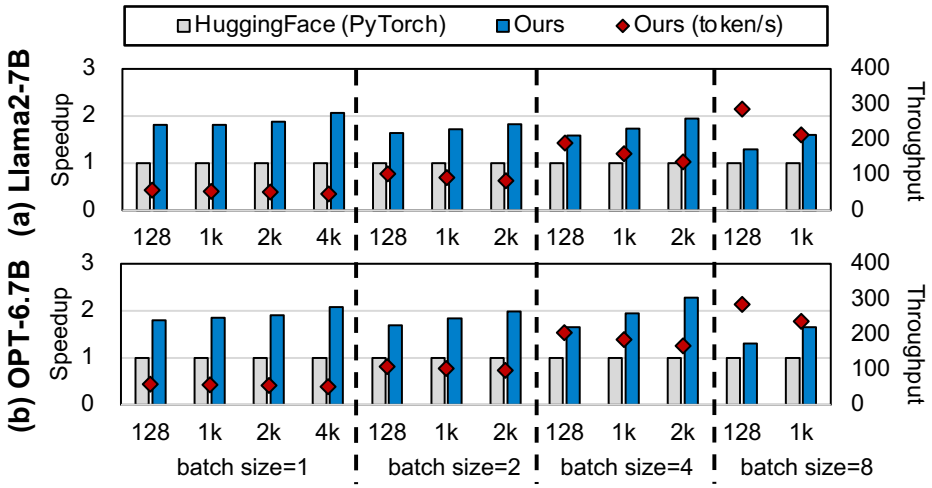

Figure 12: Speedup of the *decode* phase on AMD RX7900XTX.

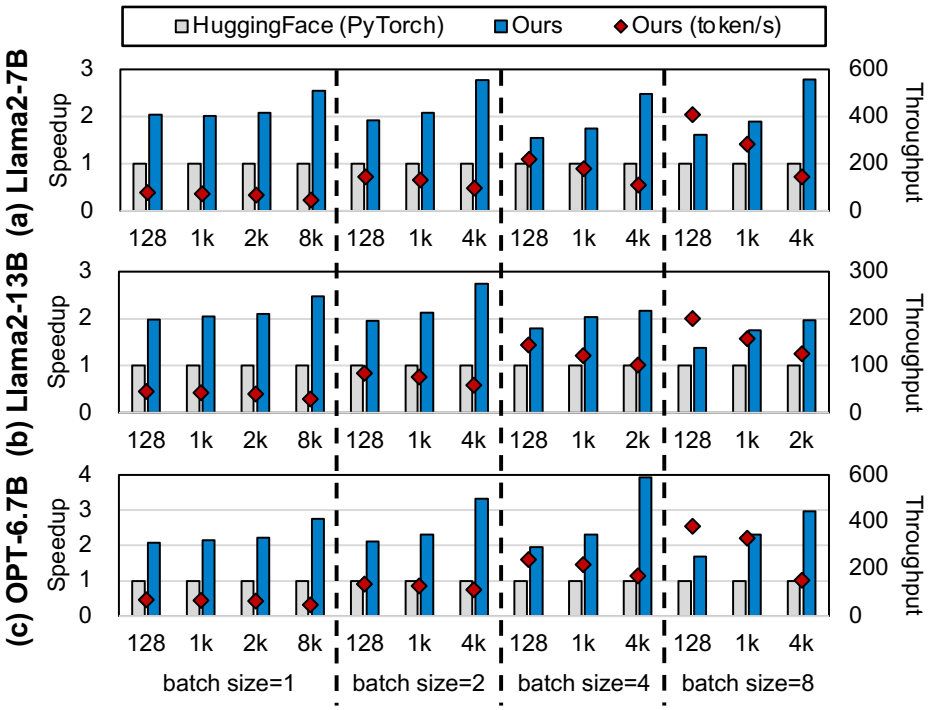

Figure 13: Speedup of the *decode* phase on AMD MI210.

8 Conclusion

We propose *FlashDecoding++*, a fast Large Language Model inference engine in this paper. *FlashDecoding++* accelerates mainstream LLMs with multiple hardware backend support. *FlashDecoding++* proposes three novel designs: the asynchronized softmax with unified max value, the flat GEMM optimization with double buffering, and the heuristic dataflow with hardware resource adaption, achieving up to **4.86**× and **3.93**× speedup on NVIDIA and AMD GPUs compared with Hugging Face implementations. *FlashDecoding++* also achieves an average of **1.37**× speedup compared with state-of-the-art LLM inference engines, FlashDecoding, on various LLMs.

References

[1] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[2] Arun James Thirunavukarasu, Darren Shu Jeng Ting, Kabilan Elangovan, Laura Gutierrez, Ting Fang Tan, and Daniel Shu Wei Ting. Large language models in medicine. *Nature medicine*, 29(8):1930–1940, 2023.

[3] Rohan Anil, Andrew M. Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, Eric Chu, Jonathan H. Clark, Laurent El Shafey, Yanping Huang, Kathy Meier-Hellstern, Gaurav Mishra, Erica Moreira, Mark Omernick, Kevin Robinson, Sebastian Ruder, Yi Tay, Kefan Xiao, Yuanzhong Xu, Yujing Zhang, Gustavo Hernandez Abrego, Junwhan Ahn, Jacob Austin, Paul Barham, Jan Botha, James Bradbury, Siddhartha Brahma, Kevin Brooks, Michele Catasta, Yong Cheng, Colin Cherry, Christopher A. Choquette-Choo, Aakanksha Chowdhery, Clément Crepy, Shachi Dave, Mostafa Dehghani, Sunipa Dev, Jacob Devlin, Mark Díaz, Nan Du, Ethan Dyer, Vlad Feinberg, Fangxiaoyu Feng, Vlad Fienber, Markus Freitag, Xavier Garcia, Sebastian Gehrmann, Lucas Gonzalez, Guy Gur-Ari, Steven Hand, Hadi Hashemi, Le Hou, Joshua Howland, Andrea Hu, Jeffrey Hui, Jeremy Hurwitz, Michael Isard, Abe Ittycheriah, Matthew Jagielski, Wenhao Jia, Kathleen Kenealy, Maxim Krikun, Sneha Kudugunta, Chang Lan, Katherine Lee, Benjamin Lee, Eric Li, Music Li, Wei Li, YaGuang Li, Jian Li, Hyeontaek Lim, Hanzhao Lin, Zhongtao Liu, Frederick Liu, Marcello Maggioni, Aroma Mahendru, Joshua Maynez, Vedant Misra, Maysam Moussalem, Zachary Nado, John Nham, Eric Ni, Andrew Nystrom, Alicia Parrish, Marie Pellat, Martin Polacek, Alex Polozov, Reiner Pope, Siyuan Qiao, Emily Reif, Bryan Richter, Parker Riley, Alex Castro Ros, Aurko Roy, Brennan Saeta, Rajkumar Samuel, Renee Shelby, Ambrose Slone, Daniel Smilkov, David R. So, Daniel Sohn, Simon Tokumine, Dasha Valter, Vijay Vasudevan, Kiran Vodrahalli, Xuezhi Wang, Pidong Wang, Zirui Wang, Tao Wang, John Wieting, Yuhuai Wu, Kelvin Xu, Yunhan Xu, Linting Xue, Pengcheng Yin, Jiahui Yu, Qiao Zhang, Steven Zheng, Ce Zheng, Weikang Zhou, Denny Zhou, Slav Petrov, and Yonghui Wu. Palm 2 technical report, 2023.

[4] Jan Clusmann, Fiona R Kolbinger, Hannah Sophie Muti, Zunamys I Carrero, Jan-Niklas Eckardt, Narmin Ghaffari Laleh, Chiara Maria Lavinia Löffler, Sophie-Caroline Schwarzkopf, Michaela Unger, Gregory P Veldhuizen, et al. The future landscape of large language models in medicine. *Communications Medicine*, 3(1):141, 2023.

[5] Can Cui, Yunsheng Ma, Xu Cao, Wenqian Ye, and Ziran Wang. Receive, reason, and react: Drive as you say with large language models in autonomous vehicles. *arXiv preprint arXiv:2310.08034*, 2023.

[6] OpenAI. Openai pricing. [Online], 2023. `https://openai.com/pricing`.

[7] Nerdynav. Up-to-date chatgpt statistics & user numbers [oct 2023]. [Online], 2023. `https://nerdynav.com/chatgpt-statistics`.

[8] AFZAL AHMAD DYLAN PATEL. The inference cost of search disruption - large language model cost analysis. [Online], 2023. `https://www.semianalysis.com/p/the-inference-cost-of-search-disruption`.

[9] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. IEEE, 2022.

[10] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Re, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu. 2023.

[11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

[12] Sensetime. Openppl: A high-performance deep learning inference platform. [Online], 2023. `https://openppl.ai/home`.

[13] Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. Flash-decoding for long-context inference. [Online], 2023. `https://crfm.stanford.edu/2023/10/12/flashdecoding.html`.

[14] Neal Vaidya, Fred Oh, and Nick Comly. Optimizing inference on large language models with nvidia tensorrt-llm, now publicly available. [Online], 2023. `https://github.com/NVIDIA/TensorRT-LLM`.

[15] Sensetime. A light and fast inference service for llm. [Online], 2023. `https://github.com/ModelTC/lightllm`.

[16] Text generation inference: Fast inference optimize for llms. [Online], 2023. https://github.com/huggingface/text-generation-inference/.

[17] Mlc llm: Machine learning compilation for large language models. [Online], 2023. https://github.com/mlc-ai/mlc-llm.

[18] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.

[19] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.

[20] Aaron Pham, Chaoyu Yang, Sean Sheng, Shenyang Zhao, Sauyon Lee, Bo Jiang, Fog Dong, Xipeng Guan, and Frost Ming. OpenLLM: Operating LLMs in production, June 2023.

[21] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. *arXiv preprint arXiv:1901.02860*, 2019.

[22] Z Dong, T Tang, L Li, and WX Zhao. A survey on long text modeling with transformers. arxiv 2023. *arXiv preprint arXiv:2302.14502*.

[23] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

[24] NVIDIA. cublas: Basic linear algebra on nvidia gpus. [Online], 2017. https://developer.nvidia.com/cublas.

[25] NVIDIA. Cutlass: Cuda templates for linear algebra subroutines. [Online], 2017. https://github.com/NVIDIA/cutlass.

[26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[27] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*, pages 807–814, 2010.

[28] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.

[29] Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017.

[30] John Bridle. Training stochastic model recognition algorithms as networks can lead to maximum mutual information estimation of parameters. *Advances in neural information processing systems*, 2, 1989.

[31] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.

[32] NVIDIA. Nvidia tensor core. [Online], 2023. https://www.nvidia.com/en-us/data-center/tensor-cores/.

[33] NVIDIA. Fastertransformer: About transformer related optimization, including bert, gpt. [Online], 2017. https://github.com/NVIDIA/FasterTransformer.

[34] Siping Wang. Fastgemv: High-speed gemv kernels. [Online], 2023. https://github.com/wangsiping97/FastGEMV.

[35] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics.

[36] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. Opt: Open pre-trained transformer language models, 2022.

[37] Zhengxiao Du, Yujie Qian, Xiao Liu, Ming Ding, Jiezhong Qiu, Zhilin Yang, and Jie Tang. Glm: General language model pretraining with autoregressive blank infilling. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 320–335, 2022.

[38] NVIDIA. Nvidia tensorrt: An sdk for high-performance deep learning inference. [Online]. https://developer.nvidia.com/tensorrt.